Universidade Leiria • Oeste
Escola de Tecnologia e Gestão

DEPARTAMENTO DE ENGENHARIA INFORMÁTICA

www.dei.estg.ipleiria.pt

Systems Integration

RESTful Web Services

Scope:
- Background
- URIs following directory structure and message contents
- Creating a RESTful Web Service
- Example and exercise
- Create a database
- CRUD operations in a Database
- Consume a RESTful API
- Document and Test a RESTful Web API service (using Swagger)

Duration: 2 classes

©2026-2027: {marisa.maximiano, nuno.costa, humberto.ferreira}@ulo.pt

Background

RESTful Web Services follow a resource-oriented architectural style for designing interoperable distributed systems. They are built on widely accepted Web standards, including URIs for resource identification, HTTP for communication, and XML or JSON for data representation. REST promotes a stateless interaction model and a uniform interface, enabling the development of scalable, maintainable, and platform-independent services.
The RESTful architectural style encourages the use of standard HTTP methods to perform operations on resources. These methods naturally correspond to the CRUD (Create, Retrieve, Update, Delete) operations typically supported by database systems, as follows:

Database Verb	**HTTP Verb**
Create	Post
Retrieve	Get
Update	Put
Delete	Delete

But is it mandatory to adopt HTTP verbs? Can we use solely GET verb and issue data retrieval, deletion, creation, and updates operations? Yes, you can use solely GET verb and use query strings to pass arguments, but this is not what RESTful defines. Proceeding by this way, we will reach a non-standard way to define services API.
Evidence of RESTful web services popularity is its adoption by World Wide Web 2.0 service providers including Facebook, Google, Yahoo, etc.
RESTful Web Services are stateless and 100% Web compatible. This means that a RESTful Web Services could be invoked directly from Browser and service scalability is completely transparent in the presence of a server cluster infrastructure.

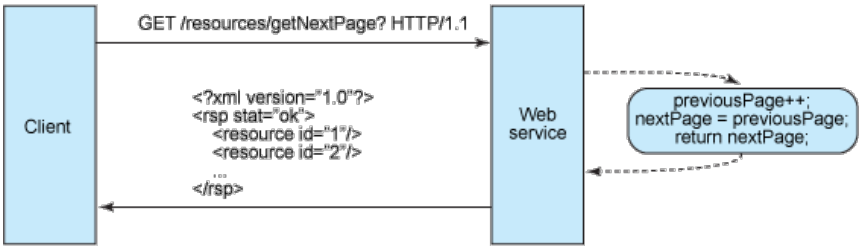


Figure 1 - Stateful design [IBM]

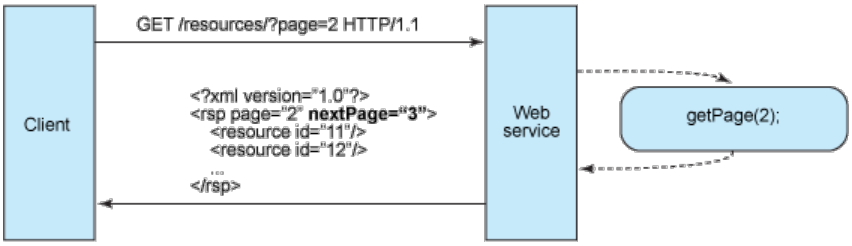


Figure 2 - Stateless design [IBM]

Here are some examples of HTTP verbs and URLs (calls) in the context of RESTful Web Services:

GET	`http://<domain>/customers`	Gets all resources
GET	`http://<domain>/customers/6`	Get resource for ID 6
POST	`http://<domain>/customers`	Creates new resource. Details in the HTTP body
PUT	`http://<domain>/customers/5`	Updates resource with ID 5. Details in the HTTP body
DELETE	`http://<domain>/customers/5`	Deletes resource with ID 5

RESTful was first described by Roy Fielding in his doctoral dissertation, *Architectural Styles and the Design of Network-based Software Architectures*:
`http://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm`
Using C# and .NET Core, RESTful Web Services can be implemented following two different approaches. The first relies on the traditional **Model–View–Controller** (MVC) pattern, where requests are handled by controllers and application logic is organized into models and actions. The second adopts a **Minimal API** approach, where HTTP endpoints are mapped directly to route handlers without requiring controllers. In both cases, the framework provides built-in support for exchanging data using standard formats such as JSON and XML, allowing resources to be exposed and consumed through HTTP in a simple and interoperable manner.

1. URIs following directory structure and message contents

To simplify client-side interactions, URIs on the server side should be structured in a way that resembles a hierarchical file systems. This approach makes resources easier to understand, locate, and access. Consider the following example of a discussion forum service [IBM]:

```
http://www.myservice.org/discussion/topics/{topic}
```

In this example, we could see a root node called discussion and a topics node beneath it. Inside topics node there can be any type of discussions threads such as database, hardware, etc. Within this structure, it's easy to pull up discussion threads just by typing something after /topics/ [IBM]. Another option to define the discussion topics could be from a date perspective [IBM]:

```
http://www.myservice.org/discussion/2019/10/01/{topic}
```

So, it is extremely important to identify all resources behind the service and structure them in a good hierarchical directory approach.

Once set up in a directory structure, the RESTful service could be invoked. For both service invocation and response, XML or JSON format could be used. XML is more suitable for high structure data while JSON (JavaScript Object Notation) is used when XML (complex) parsing must be avoided (e.g. lack of resources such as in mobile and wearable devices).

The **XML** approach:

```
<product>
     <id>1</id>
     <name>Laptop Asus</name>
     <description>ASUS ZENBOOK UX305FA. Fast. Slim. Beautiful. </description>
</product>
```

The **JSON** approach:

```
{
      "id": 1,
      "name": "Laptop Asus",
      "description": "ASUS ZENBOOK UX305FA. Fast. Slim. Beautiful."
}
```

2. Creating a RESTful Web Service (Minimal API approach)

2.1. Create a new solution named **SolutionREST**.

2.2. Create a new project named **ProductsAPI** using the **ASP.NET Core Empty** template[1]. This template is the recommended starting point for developing RESTful Web Services with the **Minimal API** approach, as it does not include controllers or other unnecessary components. The application logic is implemented by mapping HTTP endpoints directly in the `Program.cs` file, allowing resources to be exposed through simple route handlers.

The commands bellow illustrates the process of creating the project:

```
dotnet new web -n ProductsAPI
cd ProductsAPI
```

2.3. Add the project to the solution.

```
dotnet sln add ProductsAPI/ProductsAPI.csproj
```

2.4. Inspect the generated files and verify the content of each file.

Open the **`Program.cs`** file. This file contains the application's entry point and is where the RESTful Web Service endpoints will be defined using the Minimal API approach. Here are the default contents:

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/", () => "Hello World!");

app.Run();
```

2.5. Build and run the web service.

```
dotnet run
```

Open a web browser and navigate to the URL displayed in the terminal (typically `http://localhost:5000` or `https://localhost:5001`) to verify that the application is running and answering, *"Hello World!"*.

2.6. Consider the following **Product** data structure as an in-memory ("in-RAM") database and implement the HTTP endpoints for retrieving a single product (GET), retrieving all products (GET), creating a product (POST), updating a product (PUT), and deleting a product (DELETE). Using HTTP client tools (e.g., a web browser, cURL, or Postman), invoke each of the implemented endpoints.

2.6.1. Create the Product class according to the follow example, then create the in-memory data in the `Program.cs` file.

```
dotnet new class -n Product
```

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Category { get; set; }
```

[1] You may also use the **ASP .NET Core Web API** template using the `webapi` in the project creation.

```
    public decimal Price { get; set; }
}
```

```
...

List<Product> products = new List()
{
    new Product { Id = 1, Name = "Tomato Soup", Category = "Groceries", Price = 1 },
    new Product { Id = 2, Name = "Yo-yo", Category = "Toys", Price = 3.75m },
    new Product { Id = 3, Name = "Hammer", Category = "Hardware", Price = 16.99m },
};
```

2.6.2. Implement all the endpoints. Next, there is a sample implementation of the GET endpoint. In the implementation of each endpoint, return the appropriate **HTTP response code** according to the outcome of the operation.

```
app.MapGet("/api/products/{id:int}", GetProductById);

…

IResult GetProductById(int id)
{
    foreach (var product in products)
    {
        if (product.Id == id)
        {
            return Results.Ok(product);
        }
    }
    return Results.NotFound();
}

...
```

List of some of the HTTP Status Code:

HTTP Status Code	Description
200 OK	Returned for successful GET requests.
201 Created	Returned when a new resource is successfully created via a POST request.
204 No Content	Returned when a PUT or DELETE request is successful and no response body is required.
400 Bad Request	Returned when the request is invalid or contains incorrect data.
404 Not Found	Returned when the requested resource does not exist.

3. Creating a RESTful Web Service (Controllar-based approach)

3.1. Implement version 2 of the Products API using ASP.NET Core Web API controllers (controller-based API), exposing the endpoints under the **/api/v2/...** route prefix. Name this new project **ProductsAPIv2.**

```
dotnet new webapi -n ProductsAPIv2 -controllers
cd ProductsAPIv2

dotnet sln add ProductsAPIv2/ProductsAPIv2.csproj
```

The new project contains sample code in the `Controllers` folder illustrating the behaviors of several end-points. It also includes the class representing the resource used in the sample implementation.
Inspect the code and run the project to test its functionality. The `Program.cs` file should contain an implementation similar to the following example:

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
```

```
builder.Services.AddControllers();

        ...

var app = builder.Build();

        ...

app.MapControllers();

app.Run();
```

3.2. Create the `Models` folder. Inside it, create the `Product` class using the same information as in the previous API example.

3.3. Implement the controller. In the `Controllers` folder, create the `ProductsController.cs` file. The resulting project structure should be as follows:

```
Projeto
|
├── Controllers
|   └── ProductsController.cs
|
├── Models
|   └── Product.cs
|
└── Program.cs
```

The following example shows a sample implementation of the `ProductsController` class. Ensure that the controller provides endpoints to support the following CRUD operations: **Get All**, **Get by Id**, **Create**, **Update**, and **Delete**.

```
using Microsoft.AspNetCore.Mvc;
using ProductsAPIv2.Models;

namespace ProductsAPIv2.Controllers;

[ApiController]
public class ProductsController : ControllerBase
{

    static List<Product> products = new ()
    {
    new Product { Id = 1, Name = "Tomato Soup", Category = "Groceries", Price = 1 },
    new Product { Id = 2, Name = "Yo-yo", Category = "Toys", Price = 3.75m },
    new Product { Id = 3, Name = "Hammer", Category = "Hardware", Price = 16.99m },
    };

    [HttpGet("/api/v2/products")]
    public IActionResult GetAll()
    {
        return Ok(products);
    }

    // implements the other endpoint (Get by Id, Create, Update and Delete)
}
```

3.4. **[Extra/Optional]** Create a simple JavaScript client.

Now, let's call the created RESTful API using vanilla JavaScript (fetch). The current project simply needs to serve a static HTML page that uses JavaScript (fetch) to call the API.

```
mkdir wwwroot
touch wwwroot/index.html
```

Add `app.UseStaticFiles();` to the `Program.cs` file before the `app.Run();` statement.

Start by using the following HTML content as the content of index.html.

```
<!DOCTYPE html>
<html>
```

```
<head>
    <title>RESTful web service - Products App</title>
        <meta charset="utf-8" />
</head>
<body>
    <div>
        <h2>All Products</h2>
        <ul id="products"/>
    </div>
    <div>
        <h2>Search by Id</h2>
        <input type="text" id="prodId" size="5"/>
        <input type="button" value="Search" onclick="find();"/>
        <p id="product" />
    </div>


    <script>
        var uri = 'http://localhost:5000/api/v2/products'; //TODO: MUST BE UPDATED

        document.addEventListener('DOMContentLoaded', function () {
            fetch(uri)
                .then(function (response) { return response.json(); })
                .then(function (data) {
                    data.forEach(function (item) {
                        var li = document.createElement('li');
                        li.textContent = formatItem(item);
                        document.getElementById('products').appendChild(li);
                    });
                })
                .catch(function (error) {
                    alert('Request failed: ' + error.message);
                });
        });

        function formatItem(item)
        {
            return item.name + ": " + item.price + "€";
        }

        function find()
        {
            document.getElementById('product').textContent = "";
            var id = document.getElementById('prodId').value;

            fetch(uri + '/' + id)
                .then(function (response) {
                    if (response.status === 404) { alert("Product not found."); return null; }
                    if (!response.ok) { alert("HTTP " + response.status); return null; }
                    return response.json();
                })
                .then(function (data) {
                    if (data) {
                        document.getElementById('product').textContent = formatItem(data);
                    }
                })
                .catch(function (error) {
                    alert("Request failed: " + error.message);
                });
        }
    </script>
</body>
</html>
```

3.5. Besides the previous **GET** route "`/api/v2/products/1`", implement a new **action** to the controller that will allow to request all the products from specific category.

3.5.1. The string "`api/v2/products/{category}`" is the new URI template for the route. Notice that the "`{category}`" parameter in the route template matches the name of the *category* value. Web API tries to match the request URI to the template. In this example, "products" is a literal segment, and "{category}" is a variable parameter. The following URIs would match this template.
http://<domain>/api/v2/products/Toys.

```
[HttpGet("/api/v2/products/{category}")]
```

3.5.2. **[Extra]** Update also the index.html form do call the newly implemented operation.

4. Creating a RESTful Web Service with Database Access

4.1. Create a new version (version 3) of the Products RESTful API. In this version, the API will persist its data in a **SQLite** database instead of using an in-memory collection. Create a new **ASP.NET Core Web API** project named **ProductsDatabaseAPI** and implement the same RESTful endpoints developed in the previous exercise.

4.1.1. Create a new ASP.NET Core Web API project named **ProductsDatabaseAPI**.

```
dotnet new web -n ProductsDatabaseAPI
cd ProductsDatabaseAPI
code .

mkdir Controllers
touch Controllers/ProductsController.cs
```

Code in the **`ProductsController.cs`**

```
using Microsoft.AspNetCore.Mvc;

namespace ProductsAPI.Controllers;

[ApiController]
public class ProductsController : ControllerBase
{

    …
```

4.1.2. Add support for SQLite and create a database named **Products.db**.

```
dotnet add package Microsoft.Data.Sqlite
```

Create **ProductsDatabase** class to encapsulate all SQLite operations:

```
using Microsoft.Data.Sqlite;

using ProductsDatabaseAPI.Models;
public class ProductsDatabase
{
    private readonly string connectionString = "Data Source=Products.db";
    public ProductsDatabase()
    {
        CreateDatabase();
    }

    private void CreateDatabase()
    {
        using SqliteConnection connection = new SqliteConnection(connectionString);
        connection.Open();

        SqliteCommand command = connection.CreateCommand();

        command.CommandText =
        @"
        CREATE TABLE IF NOT EXISTS Products
        (
            Id INTEGER PRIMARY KEY AUTOINCREMENT,
            Name TEXT NOT NULL,
            Category TEXT NOT NULL,
            Price REAL NOT NULL
        );
        ";

        command.ExecuteNonQuery();
```

```
    }

    ...
```

4.1.3. Create the **Product** class model.

4.1.4. Implement the RESTful endpoints (GET, GET by Id, POST, PUT, and DELETE) using the SQLite database. Data access will be performed through the SQLite library, which executes SQL statements directly on the database without using the ORM framework.

4.1.5. Test each implemented RESTful endpoints using a web browser, cURL, or Postman.

4.2. A good **API documentation** is critical to provide a positive developer experience. API documentation is written text (or reference manual) that complements an API and explains how to effectively use the API. It can be created manually or automatically generated, using API documentation software.

Swagger is a simple but powerful representation of the RESTful API. Nowadays most developers are using Swagger in almost every modern programming language and deployment environment to document.

4.2.1. Add Swagger to your Web API project. Install the following package and register the swagger service in `Program.cs`, before `Build()`:

```
dotnet add package Swashbuckle.AspNetCore
```

```
builder.Services.AddSwaggerGen();
```

Before `.MapControllers()` add:

```
app.UseSwagger();
app.UseSwaggerUI();
```

4.2.2. Run the application and access the swagger documentation:

```
dotnet run
```

Once the application is running, open a web browser and navigate to:
http://localhost:5000/**swagger**

The Swagger UI will display the available API endpoints and allow you to test them interactively. Expected result:

Note: See how to personalize the generated documentation guideline here: https://learn.microsoft.com/en-us/aspnet/core/tutorials/web-api-help-pages-using-swagger.

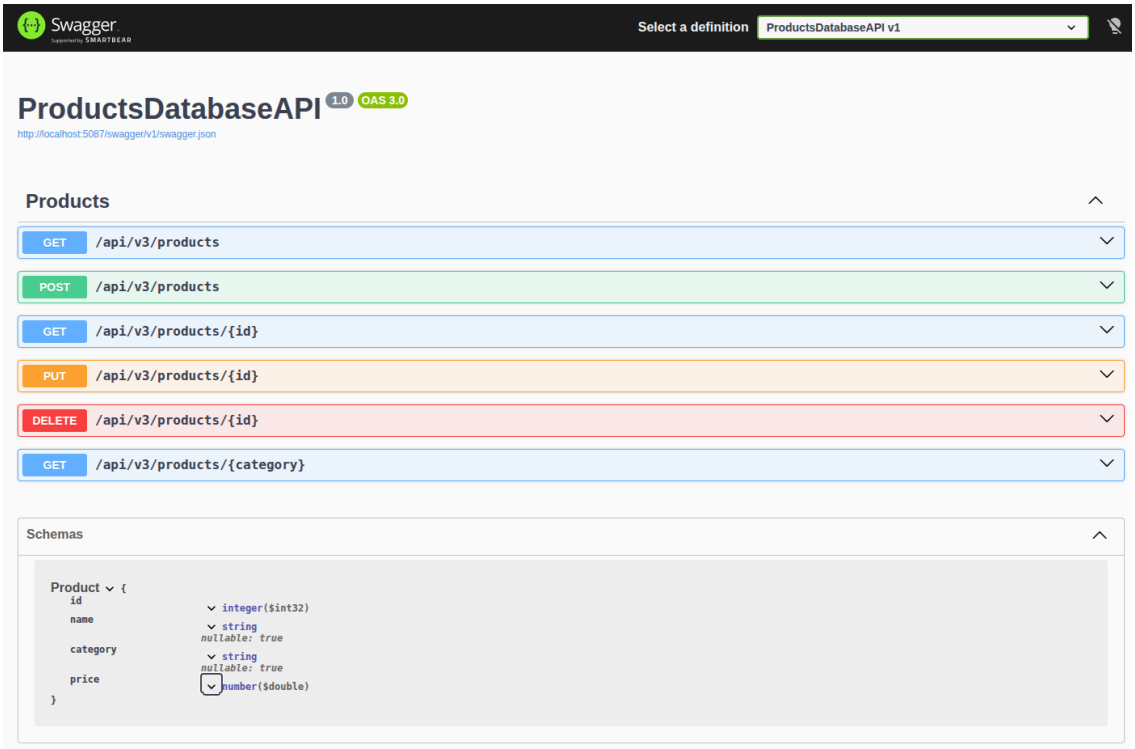


Figure 3 - Swagger documentation generated for the RESTful API

5. Creating the Client Application

Create a desktop client application named **ProductsClientApp** using the **Avalonia .NET** template. The application will consume the RESTful API developed in the previous section.

5.1. Create the project and add the project to the existing solution. Build and run the application to verify that it has been created successfully.

```
dotnet new install Avalonia.Templates

dotnet new avalonia.app -n ProductsClientApp

cd ProductsClientApp

code .

dotnet run
```

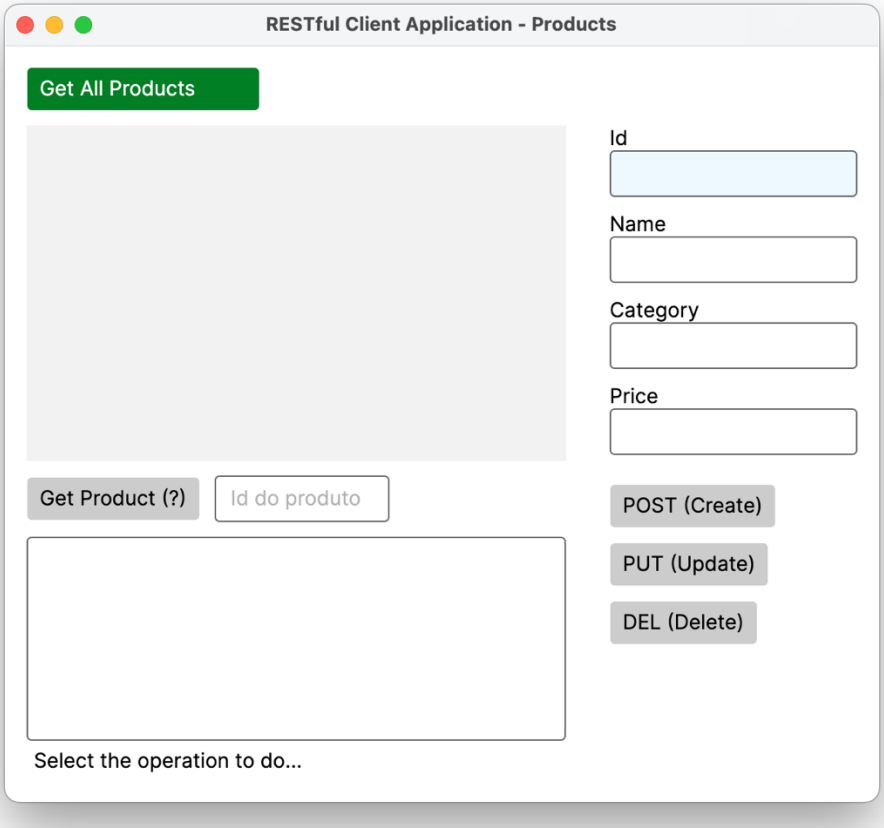


Figure 4 - Suggested interface for the ProductsClientApp application.

Replace the existing content of the `MainWindow.axaml` file with the following code:

```
< <Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        mc:Ignorable="d" d:DesignWidth="800" d:DesignHeight="450"
        x:Class="ProductsClientApp.MainWindow"
        Width="600"
        SizeToContent="Height"
        Title="RESTful Client Application - Products">
    <Grid Margin="15">

        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="2*" />
            <ColumnDefinition Width="*" />
        </Grid.ColumnDefinitions>

        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="250"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="150"/>
            <RowDefinition Height="Auto"/> <!-- Status -->
        </Grid.RowDefinitions>

        <!-- Get All -->
        <Button
            Grid.Row="0"
            Grid.Column="0"
            Name="btnGetAll"
            Content="Get All Products"
            Width="160"
            Background="#2E7D32"
```

```
            Foreground="White"
            HorizontalAlignment="Left"
            Click="GetAll_Click"/>

        <!-- List of Products -->
        <ListBox
            Grid.Row="1"
            Grid.Column="0"
            Name="lstProducts"
            Margin="0,10,10,10"/>

        <!-- Get Specific Product -->
        <StackPanel
            Grid.Row="2"
            Grid.Column="0"
            Orientation="Horizontal">

            <Button
                Name="btnGet"
                Content="Get Product (?)"
                Click="BtnGetSpecific_Click"/>

            <TextBox
                Name="txtSearchId"
                PlaceholderText="Id do produto"
                Width="120"
                Margin="10,0,0,0"/>
        </StackPanel>

        <!-- Result -->
        <TextBox
            Grid.Row="3"
            Grid.Column="0"
            Name="txtResult"
            Margin="0,10,10,0"
            AcceptsReturn="True"
            FontFamily="Consolas,Menlo,monospace"
            TextWrapping="Wrap"/>

        <!-- Right side -->
        <StackPanel
            Grid.Row="1"
            Grid.Column="1"
            Margin="20,10,0,0">

            <TextBlock Text="Id"/>
            <TextBox Name="txtId" Background="AliceBlue"/>

            <TextBlock Text="Name" Margin="0,10,0,0"/>
            <TextBox Name="txtName"/>

            <TextBlock Text="Category" Margin="0,10,0,0"/>
            <TextBox Name="txtCategory"/>

            <TextBlock Text="Price" Margin="0,10,0,0"/>
            <TextBox Name="txtPrice"/>

            <Button
                Name="btnPost"
                Margin="0,20,0,0"
                Content="POST (Create)"
                Click="BtnCreate_Click"/>

            <Button
                Name="btnPut"
                Margin="0,10,0,0"
                Content="PUT (Update)"
                Click="BtnGetUpdate_Click"/>

            <Button
                Name="btnDelete"
                Margin="0,10,0,0"
                Content="DEL (Delete)"
                Click="BtnDelete_Click"/>
```

```
        </StackPanel>
        <Border Grid.Row="4"
                Grid.ColumnSpan="2"
                Padding="5">

            <TextBlock Name="txtStatus"
                       VerticalAlignment="Center"/>
        </Border>
    </Grid>
</Window>
```

5.2. Implement the functionality of each user interface button so that, when clicked, it sends an HTTP request to the corresponding RESTful API endpoint and processes the returned response appropriately. Note: ou may use the **HttpClient**[2] class to make HTTP requests and handle API responses

[2] See the official documentation at https://learn.microsoft.com/en-us/dotnet/fundamentals/networking/http/httpclient